VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering (SCOPE)

Lab Internal Assessment Test – 8, 9 & 10

Programme Name: B.Tech. **Faculty Name: Dr.V.SIVAKUMAR**

Semester	Course Code	Course Title	ClassNbr	Slot	Date
Fall Semester 2026-27	BCSE302P	Database Systems Lab	VL2026270104025	L25+L26	11-09-2026

Instructions:

1. Write your **Name, Regno, Course code, Course Title, Slot** on the top of the word document.
2. Write the following in the test paper in order.
   1.Given Problem Statement / Question
   2.Query / Source code
   3.Input and Output / **Screenshot with Timestamp**
3. Sharing the project with other friends/ copying projects from the other class or slot students are considered as **MALPRACTICE**.
4. Upload your program before endline of time in upload link, **no extension** will be given.
5. Upload file format in **PDF only in VTOP, don't modify later then it won't consider.**
6. Upload relevant files like PDF & DOC file in **Moodle**
7. In between the **code use your Registerno & name** as variable name, method name, class name and while printing the values **(Mandatory)**
8. Your file should contain your questions, complete code (as text format) and screenshots of your results (error/output)
9. Your PDF File name should be
   REGISTERNO _SLOT_IA9&10_Proj_NAME

   Example: 25BDS0003_L25_ IA9&10_Proj _GANESH

Project Requirement

1. Problem Statement

Students shall identify a real-world database application from faculty-approved domain. The selected problem should demonstrate how a relational database system can efficiently store, manage, retrieve, secure, and analyze organizational data. The project should emphasize database design, normalization, data integrity, secure access, and modern database application development practices.

2. Project Planning

Students shall prepare a concise project proposal containing:

1. Project Title
2. Problem Statement
3. Objectives
4. Scope
5. Team Members and Responsibilities
6. Technology Stack
7. Major Deliverables
8. Project Timeline (Simple Gantt Chart)

3. Database Design Requirements

Students must design the database before implementation.

Mandatory Requirements

- Draw an Entity Relationship (ER) Diagram**.**
- Create a Relational Schema from the ER Diagram.
- Prepare a Data Dictionary (Recommended).

The database design must satisfy the following:

- Minimum 4 primary entities
- Minimum 3 relationships
- Minimum 8 database tables
- Every table shall be derived from the ER Diagram.
- Every table shall contain a Primary Key.
- Appropriate Foreign Keys shall be defined.
- Apply appropriate constraints (NOT NULL, UNIQUE, CHECK, DEFAULT, etc.).
- Normalize all tables to Third Normal Form (3NF) or higher.
- Create indexes wherever required to improve query performance.

Note: Database tables created without a corresponding ER Diagram will not be considered for evaluation.

4. Database Implementation

Students must implement the database using any free relational database management system (RDBMS).

Recommended databases include:

- PostgreSQL
- MySQL
- MariaDB
- SQLite
- MongoDB (NoSQL)
- Firebase Firestore
- CouchDB
- Cassandra

The implementation shall include:

- Database Creation
- Minimum 8–10 related tables
- CRUD Operations
- Joins
- Aggregate Queries
- Views
- Transactions
- At least one Stored Procedure *(where supported by the selected database)*
- At least one Trigger *(where supported by the selected database)*
- Appropriate indexes for improving query performance

Database interaction must be performed using an Object Relational Mapping (ORM) supported by the selected backend framework.

Database interaction shall be implemented using any free ORM**.**

Examples

Backend Technology	**Recommended Free ORM**
Spring Boot (Java)	Hibernate / Spring Data JPA
Express.js (Node.js)	Prisma, Sequelize, TypeORM
Django (Python)	Django ORM
Flask (Python)	SQLAlchemy
ASP.NET Core (C#)	Entity Framework Core
Laravel (PHP)	Eloquent ORM
PHP (Standalone)	PHP (Standalone)

Note: Students shall use an ORM for all database interactions wherever applicable. Writing raw SQL is permitted only when necessary for advanced database features (e.g., stored procedures, triggers, or performance-critical queries).

5. Application Development Requirements

Develop the application using modern development frameworks.

Example technologies include:

Frontend

- React
- Angular
- Vue
- Next.js

Backend

- Spring Boot
- Express.js
- Django
- Flask

The application shall include:

- Responsive UI
- Login page
- Dashboard
- CRUD operations
- Search
- Filtering
- Pagination
- Form validation
- Exception handling

We are in 2026, I would also recommend specific stacks so you don't struggle with incompatible technologies. This is for your reference.

Frontend	**Backend**	**Database**	**ORM**	**Authentication**
React	Spring Boot	PostgreSQL	Hibernate/JPA	Google OAuth / JWT
React	Express.js	PostgreSQL	Prisma	Google OAuth / JWT
Next.js	Express.js	PostgreSQL	Prisma	Auth.js (NextAuth)

Vue.js	Django	PostgreSQL	Django ORM	Django Authentication
Angular	ASP.NET Core	PostgreSQL	Entity Framework Core	ASP.NET Identity

6. Authentication and Role-Based Authorization

Authentication

The application shall implement secure user authentication using any one of the following mechanisms**:**

OAuth Authentication (Recommended)

Students may integrate authentication using:

- Google OAuth 2.0
- GitHub OAuth 2.0

Other Supported Methods

- JWT Authentication
- Session-based Authentication
- OpenID Connect (OIDC)

Students must demonstrate:

- User login
- User logout
- Secure session/token handling
- Retrieval of authenticated user information

Authorization and Role-Based Authorization (RBAC)

The application shall implement:

- Minimum **2 user roles**
- Role-based menu access
- Role-based page access
- Role-based API access

Examples

- Admin / User
- Faculty / Student
- Customer / Seller

Important Database Requirement where all students do mistake here

Maintain all users in a single Users table**.**

Do NOT create separate tables such as:

- Admin
- Student
- Faculty
- Manager
- Employee

Use a Role **or** RoleID column to distinguish user permissions.

Recommended Authorization Libraries

- Spring Security
- ASP.NET Identity
- Passport.js
- Auth.js
- Django Authentication

7. Database Security

Students shall implement

- BCrypt *(Recommended)*
- Argon2 *(Recommended)*

SQL Injection Prevention

- Hibernate
- Prisma
- Sequelize
- SQLAlchemy

Secrets

- .env
- dotenv

Do **NOT** hardcode

- Database passwords
- API Keys
- JWT Secrets

8. Database Backup & Recovery

Backup msut be performed using the native tools of the selected database.

PostgreSQL
- pg_dump
- pg_restore

MySQL
- mysqldump
- mysql

MariaDB
- mariadb-dump

SQLite

- sqlite3 .backup

9. Version Control

Students shall maintain the project using GitHub.

Mandatory Requirements

- One private repository
- Add all team members as collaborators
- Minimum 10 meaningful commits
- Minimum 2 branches
    - main
    - dev
- Meaningful commit messages
- Pull Requests *(Recommended)*
- README.md containing:
    - Project description
    - Installation steps
    - Technology stack
    - Execution steps
- Sensitive files containing secrets or credentials (e.g., .env, API keys, database credentials, service account keys, certificates, and private keys) must be excluded from version control by adding them to the .gitignore file before committing the project to GitHub.
- Secrets or credentials must not be hardcoded in the source code or committed to the repository.

10. Deployment

Students may deploy the application using any free cloud platform.

Recommended

- Render
- Railway
- Vercel
- Netlify
- Fly.io
- Koyeb

Students shall submit

- Live URL

Marks Split up

Section	Evaluation Criteria	Marks
1. Problem Statement & Project Planning	Project title, problem statement, objectives, team responsibilities, technology stack, deliverables, Gantt chart	2
2. Database Design	ER Diagram, minimum 4 primary entities, minimum 3 relationships, relational schema, keys, constraints, normalization (3NF or above)	4
3. Database Implementation	Database creation, table design, CRUD, joins, views, aggregate queries, transactions, stored procedure/trigger (where supported), ORM integration	7
4. Application Development	Framework-based application, dashboard, search/filtering, validation, exception handling, responsive UI	4
5. Authentication, Authorization & Database Security	OAuth/JWT/Session authentication, RBAC using a single Users table, ORM-based authorization, password hashing, SQL injection prevention, secrets management	5
6. Professional Development Practices	GitHub (progressive commits, branching, `.gitignore`), database backup & restore, Docker, Docker Hub, cloud deployment	5
Other	CI/CD Pipeline using GitHub Actions or GitLab CI/CD	1
	Deployment using a Custom Domain	1
	Any one additional industry best practice (e.g., Managed Cloud Database, Monitoring with Grafana/Prometheus, Automated Testing, Audit Logging, Performance Optimization)	1
Total		30

Other Marks (Up to +3)

Students may additionally implement

CI/CD

- GitHub Actions
- GitLab CI/CD

Monitoring

- Grafana
- Prometheus

Cloud Database

- Supabase PostgreSQL
- Railway PostgreSQL
- Aiven PostgreSQL

Custom Domain

- DuckDNS, any other